

HARTSEER BACKEND DOCUMENTATION

CAPITULO 1 - DISEÑO DEL BACKEND

INTRODUCCIÓN

El presente documento tiene como finalidad describir el diseño arquitectónico y los principios de implementación adoptados durante el desarrollo del backend de **Hartseer Analytics**, documentando las decisiones técnicas que dieron origen a su arquitectura y las responsabilidades asignadas a cada uno de los componentes que conforman el sistema.

Tras la finalización del modelo de datos, la implementación de la base de datos relacional, el desarrollo de las consultas analíticas, la construcción de los dashboards en Power BI y la posterior implementación del frontend de la aplicación, la siguiente etapa del proyecto consiste en incorporar una API REST como capa de integración entre la interfaz de usuario y la base de datos. Esta nueva arquitectura reemplaza la utilización temporal de archivos JSON empleados durante el desarrollo del frontend, permitiendo que la información sea obtenida de forma dinámica, segura y centralizada.

A diferencia de la arquitectura utilizada durante las primeras etapas del proyecto, donde parte del procesamiento analítico era ejecutado en el navegador debido a la ausencia de un backend, la incorporación de una API permite trasladar toda la lógica de negocio al servidor. De esta manera, el frontend deja de ser responsable del procesamiento de la información y pasa a desempeñar exclusivamente funciones relacionadas con la presentación e interacción con el usuario, mientras que el backend concentra el acceso a los datos, la validación de solicitudes, la ejecución de reglas de negocio y la construcción de las respuestas consumidas por la aplicación.

Durante el proceso de diseño se adoptó una arquitectura basada en capas, donde cada componente posee una única responsabilidad claramente definida. Esta separación de responsabilidades favorece la mantenibilidad del sistema, reduce el acoplamiento entre módulos y facilita la evolución de la aplicación a medida que nuevas funcionalidades sean incorporadas en futuras versiones.

Del mismo modo, el diseño de la API fue concebido siguiendo una filosofía orientada al dominio del negocio y no a la interfaz de usuario. En consecuencia, los contratos

definidos por cada endpoint representan entidades y conceptos propios del negocio, evitando que la estructura de las respuestas dependa de componentes visuales específicos del frontend. Esta decisión permite que la misma API pueda ser reutilizada por diferentes consumidores, como la aplicación web, futuras integraciones con Power BI, servicios de exportación, sistemas de monitoreo u otras aplicaciones que requieran acceder a la información analítica del proyecto.

Al igual que en el resto de la documentación técnica de **Hartseer Analytics**, este documento no tiene como objetivo describir la implementación detallada del código fuente, sino documentar las decisiones arquitectónicas adoptadas durante el diseño del backend, explicando el propósito de cada capa, las responsabilidades que asume dentro del sistema y los criterios que justifican su incorporación. De esta manera, la documentación constituye una referencia conceptual de la arquitectura implementada, facilitando tanto su mantenimiento como su evolución en futuras versiones del proyecto.

OBJETIVOS DEL BACKEND

El desarrollo del backend de **Hartseer Analytics** tiene como objetivo incorporar una capa intermedia entre la interfaz de usuario y la base de datos, responsable de centralizar el procesamiento de la información y administrar el acceso a los datos del sistema. Su incorporación responde a la necesidad de reemplazar la arquitectura temporal basada en archivos JSON utilizada durante el desarrollo del frontend por una solución capaz de ofrecer un acceso seguro, escalable y mantenible a la información analítica del proyecto.

Uno de los principales objetivos de esta arquitectura consiste en desacoplar completamente el frontend de la base de datos relacional. Bajo este enfoque, la interfaz de usuario deja de interactuar directamente con la estructura de MySQL, consumiendo únicamente información procesada y estructurada mediante una API REST. Esta separación permite que ambas capas evolucionen de manera independiente, reduciendo el acoplamiento entre componentes y facilitando futuras modificaciones sin afectar el funcionamiento general de la aplicación.

Del mismo modo, el backend asume la responsabilidad exclusiva de ejecutar toda la lógica de negocio asociada al proyecto. Las validaciones de entrada, el procesamiento de métricas, la aplicación de reglas analíticas, la agregación de información y la construcción de las respuestas dejan de formar parte del frontend para concentrarse en una única capa especializada. Esta decisión garantiza que toda la información consumida por la aplicación sea generada bajo un conjunto único de reglas, evitando

duplicidad de lógica y asegurando la consistencia de los resultados independientemente del cliente que consuma la API.

Otro objetivo fundamental consiste en proteger el acceso a la información almacenada en la base de datos. La implementación de una API elimina la necesidad de exponer directamente la estructura interna del modelo relacional, permitiendo controlar el acceso mediante endpoints definidos específicamente para cada necesidad del negocio. De esta manera, el backend actúa como un mecanismo de abstracción que expone únicamente la información necesaria para cada operación, preservando la integridad y seguridad de los datos.

La arquitectura también fue diseñada con un fuerte enfoque en la reutilización y escalabilidad. En lugar de construir respuestas orientadas exclusivamente a los componentes visuales del frontend, los contratos definidos por la API representan entidades y conceptos propios del dominio del negocio. Este enfoque permite que la misma información pueda ser reutilizada por distintos consumidores, como futuras aplicaciones móviles, integraciones con Power BI, sistemas de exportación de reportes, módulos de monitoreo o cualquier otro servicio que requiera acceder a la información analítica del proyecto sin necesidad de modificar la lógica existente.

Finalmente, el backend busca establecer una base arquitectónica preparada para la evolución futura de **Hartseer Analytics**. La adopción de una arquitectura basada en capas, la definición anticipada de contratos estables y la separación estricta de responsabilidades permiten incorporar nuevas funcionalidades, ampliar la cantidad de endpoints o integrar nuevos consumidores sin comprometer la compatibilidad ni la mantenibilidad del sistema. En consecuencia, el backend deja de ser únicamente un mecanismo de comunicación con la base de datos para convertirse en el núcleo responsable de exponer, proteger y administrar toda la lógica de negocio de la aplicación.

ARQUITECTURA GENERAL

La arquitectura del backend de **Hartseer Analytics** fue diseñada siguiendo un modelo basado en capas, donde cada componente asume una responsabilidad específica dentro del flujo de procesamiento de una solicitud. Este enfoque tiene como objetivo separar claramente las distintas responsabilidades del sistema, reduciendo el acoplamiento entre módulos y favoreciendo la mantenibilidad, escalabilidad y reutilización del código a medida que el proyecto evolucione.

A diferencia de una arquitectura donde la interfaz de usuario interactúa directamente con la base de datos o donde la lógica de negocio se encuentra distribuida entre múltiples componentes, el backend concentra todo el procesamiento en una única capa de servicios, permitiendo que cada solicitud siga un recorrido predefinido hasta obtener la información requerida. De esta manera, el sistema mantiene un flujo de ejecución uniforme para todos los endpoints, independientemente del tipo de información solicitada.

La comunicación entre el frontend y la base de datos se realiza exclusivamente a través de una API REST, la cual actúa como punto de entrada único para todas las operaciones del sistema. Cada solicitud enviada por el cliente atraviesa una secuencia de componentes especializados, donde la petición es validada, procesada, consultada y transformada antes de construir la respuesta que será devuelta al consumidor.

Conceptualmente, el flujo general de procesamiento puede representarse de la siguiente manera:

Frontend

↓

Middleware Pipeline

↓

Router

↓

Controller

↓

Service

↓

Repository

↓

MySQL

Dentro de esta arquitectura, cada capa posee una única responsabilidad claramente definida. El **Middleware Pipeline** intercepta todas las solicitudes entrantes para ejecutar procesos transversales antes de que la petición continúe su recorrido. El **Router** identifica el endpoint solicitado y determina qué componente debe procesar la operación. Posteriormente, el **Controller** coordina la ejecución de la solicitud delegando toda la lógica de negocio al **Service**, quien concentra el procesamiento analítico, la aplicación de reglas del negocio y la construcción de la información que será consumida por la aplicación.

Para obtener los datos necesarios, el **Service** interactúa exclusivamente con la capa **Repository**, responsable de aislar completamente el acceso a la base de datos. Esta decisión evita que el resto de la aplicación conozca detalles relacionados con consultas SQL, estructuras relacionales o mecanismos de persistencia, permitiendo que cualquier modificación futura sobre la tecnología de almacenamiento tenga un impacto mínimo sobre las capas superiores del sistema.

Una vez recuperada la información desde MySQL, el flujo se ejecuta en sentido inverso. El **Repository** devuelve los datos al **Service**, donde son procesados, agregados y transformados en estructuras orientadas al dominio del negocio. Posteriormente, el **Controller** construye la respuesta HTTP correspondiente, la cual es serializada en formato JSON antes de ser enviada nuevamente al frontend.

La adopción de esta arquitectura responde a una decisión de diseño orientada a largo plazo. La separación estricta de responsabilidades permite que cada componente evolucione de manera independiente, facilita la incorporación de nuevas funcionalidades y simplifica las tareas de mantenimiento, depuración y pruebas del sistema. Al mismo tiempo, esta organización establece una base sólida para futuras integraciones, como aplicaciones móviles, herramientas de Business Intelligence, servicios de exportación o sistemas de monitoreo, los cuales podrán consumir la misma API sin necesidad de modificar la lógica de negocio existente.

FLUJO DE PETICIONES

Toda interacción entre el frontend y la base de datos sigue un flujo de procesamiento previamente definido, donde cada solicitud recorre una secuencia de componentes especializados antes de obtener una respuesta. Este recorrido garantiza que todas las operaciones del sistema sean procesadas bajo una misma arquitectura, independientemente del endpoint solicitado, manteniendo un comportamiento uniforme y una clara separación de responsabilidades entre las distintas capas que conforman el backend.

El ciclo comienza cuando el frontend realiza una petición HTTP hacia uno de los endpoints expuestos por la API. Antes de que dicha solicitud sea procesada, atraviesa el **Middleware Pipeline**, encargado de ejecutar aquellos procesos transversales que deben aplicarse a todas las peticiones del sistema, como la generación de identificadores de solicitud, el registro de eventos, la validación inicial o cualquier otra funcionalidad compartida por toda la aplicación.

Una vez superada esta etapa, la solicitud es recibida por el **Router**, quien identifica el endpoint solicitado y determina qué controlador será responsable de gestionar la operación. A partir de ese momento, el **Controller** actúa como coordinador del flujo de ejecución, delegando el procesamiento de la lógica de negocio al **Service**, capa donde se concentran todas las reglas analíticas implementadas por la aplicación.

Cuando la lógica requiere acceder a la información almacenada en la base de datos, el **Service** solicita los datos necesarios al **Repository**, responsable exclusivo de establecer la comunicación con MySQL. Esta separación evita que las capas superiores dependan directamente del modelo relacional o de las consultas SQL, favoreciendo una arquitectura desacoplada y fácilmente mantenible.

Una vez recuperada la información, el flujo continúa en sentido inverso. Los datos regresan al **Service**, donde son procesados, transformados y organizados de acuerdo con los contratos definidos para cada endpoint. Posteriormente, el **Controller** construye la respuesta HTTP correspondiente y la API serializa el contenido en formato JSON antes de enviarlo nuevamente al frontend para su visualización.

Este modelo de procesamiento permite que toda la lógica de negocio permanezca centralizada en el backend, mientras que el frontend asume exclusivamente responsabilidades relacionadas con la presentación de la información. Como consecuencia, cualquier consumidor de la API obtiene resultados consistentes, independientemente de la plataforma desde la cual acceda a los datos, manteniendo un comportamiento homogéneo en toda la aplicación.

RESPONSABILIDADES DE CADA CAPA

La arquitectura del backend de **Hartseer Analytics** se encuentra organizada mediante una estructura basada en capas, donde cada componente asume una responsabilidad única y claramente delimitada dentro del flujo de procesamiento de una solicitud. Esta organización responde al principio de **Responsabilidad Única (Single Responsibility Principle)**, buscando que cada módulo del sistema conozca únicamente aquellas tareas que le corresponden, evitando dependencias innecesarias y reduciendo el acoplamiento entre las distintas partes de la aplicación.

En lugar de concentrar toda la lógica del sistema en un único componente, el procesamiento de una petición se distribuye entre diversas capas especializadas que colaboran entre sí siguiendo un flujo predefinido. Cada una de ellas resuelve un problema específico dentro de la arquitectura y delega las responsabilidades restantes a la capa correspondiente. Como consecuencia, la modificación de una determinada funcionalidad suele afectar únicamente al componente responsable de dicha tarea, simplificando considerablemente las labores de mantenimiento, pruebas y evolución del sistema.

Esta separación también favorece la reutilización de la lógica de negocio y permite que la API mantenga un comportamiento homogéneo en todos sus endpoints. Independientemente del recurso solicitado, cada petición recorrerá exactamente la misma estructura arquitectónica, garantizando consistencia en el procesamiento, facilidad para incorporar nuevas funcionalidades y una organización del código alineada con buenas prácticas de desarrollo de software.

A continuación, se describen las responsabilidades asumidas por cada una de las capas que conforman la arquitectura del backend de **Hartseer Analytics**, explicando el propósito de su incorporación y la función que desempeña dentro del flujo general de procesamiento.

4.1 Middleware Pipeline

Creo que este apartado merece especial atención porque, además de explicar qué hace, también debemos justificar **por qué decidimos implementar un pipeline** en lugar de ejecutar estas tareas directamente dentro de los controladores. Esa decisión arquitectónica fue una de las primeras que tomamos para el backend y vale la pena dejar documentado el razonamiento. A partir de ahí seguiríamos con **Router**, **Controller**, **Service**, **Repository** y **Database**, manteniendo exactamente la misma estructura y profundidad en cada una de las subsecciones.

4.2 Router

La capa **Router** constituye el punto de entrada lógico de la API y tiene como responsabilidad identificar cada solicitud recibida por el sistema, asociándola con el endpoint correspondiente. Su función consiste en actuar como un mecanismo de enrutamiento que determina qué controlador deberá procesar la operación solicitada, estableciendo el primer nivel de organización funcional dentro de la aplicación.

La incorporación de una capa dedicada exclusivamente al enrutamiento responde a la necesidad de separar la definición de las rutas del resto de la lógica del sistema. Bajo esta arquitectura, el Router no ejecuta validaciones de negocio, no accede a la base de datos ni procesa información analítica. Su única responsabilidad consiste en asociar una dirección URL y un método HTTP con el componente encargado de gestionar la solicitud.

Esta separación permite que la estructura de la API permanezca organizada y fácilmente escalable a medida que nuevos recursos son incorporados al sistema. Cada conjunto de endpoints puede agruparse según el dominio funcional al que pertenece, manteniendo una organización consistente y facilitando tanto la navegación del código como la incorporación de futuras versiones de la API sin afectar la lógica implementada en otras capas.

Dentro de **Hartseer Analytics**, el Router actúa como un mecanismo de coordinación inicial entre la infraestructura de la API y la lógica de negocio. Una vez identificada la operación solicitada, delega inmediatamente el control al **Controller** correspondiente, manteniendo un desacoplamiento completo entre la estructura de las rutas y el procesamiento interno de la aplicación.

Como consecuencia, cualquier modificación relacionada con la organización de los endpoints, la incorporación de nuevas versiones de la API o la creación de nuevos recursos puede realizarse sin necesidad de alterar la lógica de negocio existente. Esta decisión favorece la mantenibilidad del proyecto, mejora la claridad de la arquitectura y establece una separación precisa entre la definición de la interfaz pública de la API y los componentes responsables de implementar su funcionamiento.

4.3 Controller

La capa **Controller** constituye el punto de coordinación entre la infraestructura de la API y la lógica de negocio implementada por la aplicación. Su responsabilidad consiste en recibir las solicitudes previamente enrutadas por el **Router**, coordinar el flujo general de ejecución y delegar el procesamiento de la operación al **Service** correspondiente, sin participar directamente en la aplicación de reglas de negocio ni en el acceso a la base de datos.

Dentro de la arquitectura de **Hartseer Analytics**, el Controller actúa como un intermediario entre la interfaz pública de la API y las capas encargadas del procesamiento interno. Su función principal consiste en interpretar la solicitud recibida, invocar el servicio adecuado y construir la respuesta HTTP que será enviada al cliente una vez finalizada la operación.

Esta separación responde al principio de mantener la lógica de coordinación aislada de la lógica de negocio. Bajo este enfoque, el Controller no realiza cálculos analíticos, no ejecuta consultas SQL ni implementa validaciones propias del dominio del negocio. Todas estas responsabilidades son delegadas al **Service**, permitiendo que el Controller permanezca liviano, predecible y fácilmente mantenible.

La adopción de este patrón aporta una importante ventaja desde el punto de vista arquitectónico. Al limitar el Controller exclusivamente a tareas de coordinación, cualquier modificación en las reglas de negocio puede realizarse dentro de la capa de servicios sin afectar la estructura de los endpoints ni la forma en que la API expone sus recursos. Del mismo modo, la lógica implementada por un mismo servicio puede ser reutilizada por distintos controladores o consumidores internos sin generar duplicidad de código.

Como consecuencia, el Controller se convierte en un componente cuya única responsabilidad consiste en gestionar el ciclo de vida de una solicitud dentro de la API, coordinando la comunicación entre las distintas capas sin asumir responsabilidades que pertenecen al dominio del negocio. Esta decisión favorece una arquitectura desacoplada, facilita la realización de pruebas unitarias y mantiene una clara separación entre la definición de la interfaz pública de la API y la implementación de las reglas analíticas que sustentan el funcionamiento de **Hartseer Analytics**.

4.4 Service

La capa **Service** constituye el núcleo funcional del backend de **Hartseer Analytics**, concentrando la totalidad de la lógica de negocio implementada por la aplicación. Su responsabilidad consiste en procesar la información obtenida desde la base de datos, aplicar las reglas analíticas definidas para cada operación y construir las estructuras de datos que posteriormente serán expuestas por la API.

A diferencia del **Controller**, cuya función se limita a coordinar el flujo de una solicitud, el **Service** es el componente responsable de interpretar el dominio del negocio. Todas las decisiones relacionadas con cálculos analíticos, agregaciones, comparaciones temporales, validaciones funcionales, construcción de indicadores y generación de métricas son ejecutadas exclusivamente dentro de esta capa, evitando que dicha lógica se distribuya entre otros componentes de la arquitectura.

Uno de los principios fundamentales adoptados durante el diseño del backend consiste en desacoplar completamente la lógica de negocio de la infraestructura de la aplicación. Como consecuencia, el **Service** no conoce detalles relacionados con la implementación de los endpoints ni con la forma en que la información será presentada por el frontend. Del mismo modo, tampoco establece comunicación directa con la base de datos, delegando el acceso a la información exclusivamente al **Repository**.

Esta separación permite que la lógica analítica permanezca completamente independiente de los mecanismos de persistencia y de la interfaz pública de la API. De esta manera, cualquier modificación sobre las reglas del negocio puede implementarse sin afectar la estructura de los controladores, los contratos de los endpoints o la tecnología utilizada para acceder a la base de datos.

Durante el diseño de **Hartseer Analytics** también se adoptó el principio de **reutilización de la información** dentro de esta capa. Siempre que resulta posible, el **Service** reutiliza los datos obtenidos mediante una única consulta al **Repository** para construir múltiples componentes de la respuesta, evitando consultas redundantes sobre la base de datos. Este enfoque permite reducir el número de operaciones ejecutadas sobre MySQL, mejorar el rendimiento general del sistema y centralizar el procesamiento analítico en un único componente especializado.

Bajo esta arquitectura, el **Service** es además responsable de construir respuestas orientadas al dominio del negocio y no a la interfaz de usuario. Los objetos devueltos por la API representan entidades analíticas reutilizables, independientemente de la forma en que posteriormente sean visualizadas por el frontend. Como consecuencia, la misma información puede ser consumida por diferentes clientes, como la aplicación web, futuras integraciones con herramientas de Business Intelligence, sistemas de

exportación de reportes o nuevos módulos incorporados en versiones posteriores del proyecto.

La incorporación de una capa de servicios responde, por lo tanto, a una decisión de diseño orientada a centralizar el conocimiento del negocio en un único punto de la arquitectura. Esta organización favorece la mantenibilidad del sistema, simplifica la incorporación de nuevas funcionalidades y garantiza que todas las reglas analíticas sean ejecutadas de forma consistente, independientemente del consumidor que acceda a la API.

4.5 Repository

La capa **Repository** constituye el único componente de la arquitectura autorizado para establecer comunicación directa con la base de datos. Su responsabilidad consiste en encapsular completamente el acceso a la información persistida, ejecutando las consultas necesarias para recuperar los datos requeridos por la lógica de negocio sin exponer detalles relacionados con la implementación del modelo relacional o del motor de base de datos utilizado.

Dentro de la arquitectura de **Hartseer Analytics**, el Repository actúa como una capa de abstracción entre el **Service** y MySQL. Bajo este enfoque, los servicios no conocen la estructura física de la base de datos, las consultas SQL implementadas ni los mecanismos utilizados para obtener la información. Su única responsabilidad consiste en solicitar los datos necesarios al Repository, delegando completamente el acceso a la persistencia.

Esta separación responde al objetivo de desacoplar la lógica de negocio de la tecnología de almacenamiento. Como consecuencia, cualquier modificación relacionada con la estructura de la base de datos, la optimización de consultas o incluso la sustitución futura del mecanismo de persistencia podrá realizarse dentro del Repository sin afectar el funcionamiento de las capas superiores de la aplicación.

A diferencia del **Service**, cuya responsabilidad consiste en interpretar la información desde una perspectiva analítica, el Repository no aplica reglas de negocio, no realiza cálculos ni construye estructuras orientadas al dominio funcional de la aplicación. Su propósito es recuperar los datos solicitados de la forma más eficiente posible y devolverlos al Service para que este continúe con el procesamiento correspondiente.

Durante el diseño de la arquitectura también se adoptó el criterio de mantener las consultas SQL centralizadas dentro de esta capa. Esta decisión evita la duplicación de código, facilita la optimización de consultas y simplifica las tareas de mantenimiento

cuando resulta necesario modificar la forma en que la información es obtenida desde MySQL. Del mismo modo, la concentración del acceso a datos en un único componente favorece la realización de pruebas unitarias y permite validar de forma independiente tanto la lógica de negocio como la persistencia de la información.

La incorporación de una capa Repository responde, por lo tanto, a una decisión orientada a preservar la independencia entre el dominio del negocio y la infraestructura de almacenamiento. Esta organización fortalece la mantenibilidad del sistema, mejora la reutilización de los mecanismos de acceso a datos y establece una arquitectura preparada para evolucionar sin que las modificaciones realizadas sobre la base de datos afecten directamente al resto de la aplicación.

4.6 Database (MySQL)

La base de datos constituye la capa de persistencia del backend de **Hartseer Analytics**, siendo responsable del almacenamiento permanente de toda la información utilizada por la aplicación. Su función consiste en conservar la integridad de los datos del negocio y proporcionar un mecanismo confiable para su consulta, actualización y administración a través de las capas superiores de la arquitectura.

Dentro del diseño del sistema, MySQL representa el nivel más bajo de la arquitectura y permanece completamente aislado del resto de los componentes mediante la capa **Repository**. Esta decisión evita que controladores, servicios o cualquier otro módulo de la aplicación establezcan comunicación directa con la base de datos, garantizando que toda interacción con la información persistida se realice de forma centralizada y controlada.

La información almacenada en MySQL corresponde al modelo relacional previamente diseñado durante las etapas iniciales del proyecto, el cual constituye la fuente única de datos sobre la que se apoyan todos los procesos analíticos implementados por la aplicación. Como consecuencia, la base de datos actúa como el repositorio oficial de la información, mientras que la API se encarga de interpretar dichos datos y transformarlos en estructuras orientadas al dominio del negocio.

Es importante destacar que la base de datos no posee conocimiento alguno sobre las reglas analíticas implementadas por la aplicación. Su responsabilidad se limita exclusivamente al almacenamiento y recuperación eficiente de la información, mientras que toda decisión relacionada con cálculos, agregaciones, comparaciones temporales o construcción de indicadores es ejecutada por la capa **Service**. Esta separación permite mantener claramente diferenciadas las responsabilidades de

persistencia y procesamiento, evitando trasladar lógica de negocio hacia el modelo de datos.

La incorporación de MySQL como mecanismo de persistencia responde a los requerimientos funcionales y analíticos del proyecto, proporcionando un sistema relacional robusto capaz de gestionar eficientemente grandes volúmenes de información estructurada y facilitar la ejecución de consultas complejas. Sin embargo, desde una perspectiva arquitectónica, la aplicación no depende directamente del motor de base de datos utilizado, sino de la abstracción proporcionada por la capa **Repository**. Esta decisión reduce el acoplamiento tecnológico y permite que futuras modificaciones sobre la infraestructura de almacenamiento puedan abordarse con un impacto mínimo sobre el resto del sistema.

En conjunto, la base de datos constituye el fundamento sobre el cual se construye toda la arquitectura analítica de **Hartseer Analytics**, proporcionando una fuente consistente y centralizada de información, mientras que las capas superiores de la aplicación se encargan de transformar dichos datos en conocimiento útil para los distintos consumidores de la API.

FILOSOFIA DE DISEÑO

El diseño del backend de **Hartseer Analytics** fue concebido bajo una filosofía orientada a la construcción de una arquitectura mantenible, desacoplada y preparada para evolucionar junto con el proyecto. Más allá de la implementación tecnológica seleccionada, todas las decisiones arquitectónicas fueron tomadas siguiendo un conjunto de principios cuyo objetivo consiste en garantizar la consistencia del sistema, facilitar su mantenimiento y permitir la incorporación de nuevas funcionalidades sin comprometer la estabilidad de la aplicación.

Uno de los principios fundamentales adoptados durante el diseño consiste en considerar al backend como el núcleo responsable de la lógica del negocio. En consecuencia, toda regla analítica, validación, procesamiento de información y construcción de indicadores es ejecutada exclusivamente dentro de la API, evitando trasladar responsabilidades al frontend. Bajo este enfoque, la interfaz de usuario deja de interpretar los datos obtenidos desde la base de datos y pasa a consumir información previamente procesada, concentrando su responsabilidad únicamente en la representación visual y la interacción con el usuario.

Del mismo modo, la arquitectura fue diseñada siguiendo una estricta separación de responsabilidades entre sus distintas capas. Cada componente posee una única

función claramente definida dentro del sistema y delega el resto de las operaciones al componente especializado correspondiente. Esta organización reduce el acoplamiento entre módulos, facilita la reutilización del código y simplifica la incorporación de nuevas funcionalidades sin afectar el comportamiento del resto de la aplicación.

Otro principio adoptado durante el proceso de diseño consiste en orientar la API al dominio del negocio y no a la interfaz de usuario. Los contratos definidos por cada endpoint representan entidades, indicadores y conceptos analíticos propios del negocio, evitando construir respuestas condicionadas por la forma en que posteriormente serán visualizadas por el frontend. Como consecuencia, una misma respuesta puede ser reutilizada por diferentes consumidores, como aplicaciones web, futuras aplicaciones móviles, herramientas de Business Intelligence, sistemas de exportación o cualquier otro servicio que requiera acceder a la información analítica del proyecto.

Con el mismo criterio, todas las respuestas de la API fueron diseñadas bajo contratos estables y homogéneos. Cada endpoint mantiene una estructura consistente para la representación de datos, metadatos, indicadores y errores, permitiendo que cualquier consumidor pueda interpretar las respuestas siguiendo un único criterio de integración. Esta decisión favorece la mantenibilidad del sistema y reduce el impacto que futuras ampliaciones puedan generar sobre los clientes que consumen la API.

Durante el diseño también se adoptó el principio de reutilización de la información procesada. Siempre que resulta posible, la lógica de negocio reutiliza los datos obtenidos mediante una única consulta a la base de datos para construir múltiples componentes de la respuesta, evitando consultas redundantes y reduciendo la carga sobre el sistema de persistencia. Esta estrategia mejora el rendimiento general de la aplicación y concentra el procesamiento analítico dentro de una única capa especializada.

Otro aspecto considerado desde las primeras etapas del diseño fue la preparación para la evolución futura del proyecto. La arquitectura fue concebida para incorporar nuevas funcionalidades mediante la extensión de la API, priorizando la compatibilidad hacia atrás y evitando modificaciones incompatibles sobre los contratos existentes. Este enfoque permite que futuras versiones de **Hartseer Analytics** puedan incorporar nuevos endpoints, ampliar la información expuesta o integrar nuevos consumidores sin comprometer el funcionamiento de las implementaciones ya existentes.

Finalmente, todas las decisiones arquitectónicas adoptadas durante el desarrollo del backend responden a una misma filosofía de diseño: construir una API que represente el conocimiento del negocio y no simplemente un mecanismo de acceso a la base de datos. Bajo esta perspectiva, la base de datos constituye la fuente de información del

sistema, mientras que la API se convierte en el componente responsable de interpretar dichos datos, aplicar las reglas analíticas correspondientes y exponer una representación consistente, reutilizable y preparada para acompañar el crecimiento futuro de **Hartseer Analytics**.

DISEÑO DE CONTRATOS

Uno de los principios fundamentales adoptados durante el diseño de la API consiste en mantener contratos de comunicación homogéneos, estables y orientados al dominio del negocio. Cada endpoint fue concebido para exponer información analítica mediante una estructura consistente, permitiendo que cualquier consumidor de la API pueda interpretar las respuestas siguiendo un único criterio de integración, independientemente del recurso solicitado.

En lugar de construir respuestas específicas para los componentes visuales del frontend, los contratos fueron diseñados para representar entidades, métricas e indicadores propios del negocio. Esta decisión permite desacoplar completamente la lógica de presentación de la estructura de la información, favoreciendo la reutilización de los datos por diferentes consumidores sin necesidad de modificar la API cuando la interfaz evolucione o incorpore nuevas visualizaciones.

Con el propósito de mantener una experiencia de integración uniforme, todos los endpoints comparten una estructura general de respuesta basada en un conjunto común de datos y metadatos. Mientras el bloque principal contiene la información analítica correspondiente a cada recurso, los metadatos proporcionan contexto adicional sobre la solicitud procesada, incluyendo información relacionada con el período analizado, la versión de la API, la generación de la respuesta y otros elementos que permiten interpretar correctamente los datos devueltos sin depender del conocimiento interno de la aplicación.

Siguiendo el mismo criterio, la representación de indicadores clave fue estandarizada mediante una estructura reutilizable para todos los dashboards del proyecto. En lugar de definir un formato diferente para cada KPI, los indicadores fueron diseñados bajo un mismo modelo conceptual, incorporando tanto el valor principal como la información necesaria para interpretar su evolución dentro del período analizado. Esta homogeneidad simplifica el consumo de la API, reduce la complejidad del frontend y favorece la incorporación de nuevos indicadores sin alterar la estructura general de los contratos existentes.

La misma filosofía fue aplicada al diseño de las colecciones utilizadas por los distintos endpoints. En lugar de construir estructuras independientes para cada gráfico o

componente visual, las respuestas agrupan la información según entidades del dominio del negocio, como productos, clientes o canales comerciales.

Posteriormente, cada consumidor decide cómo representar dicha información según sus necesidades específicas. Como consecuencia, una única colección puede alimentar múltiples visualizaciones, evitando la duplicación de datos y reduciendo la complejidad de la API.

Otro aspecto considerado durante el diseño fue la estandarización del tratamiento de errores. Todas las respuestas asociadas a condiciones excepcionales mantienen un formato común, independientemente del endpoint donde se produzcan. Esta decisión permite que los consumidores interpreten los errores de forma consistente, reaccionando principalmente a códigos de error y estados HTTP estandarizados en lugar de depender de mensajes descriptivos, favoreciendo una integración más robusta y preparada para futuras evoluciones.

Finalmente, los contratos fueron concebidos con un fuerte enfoque en la compatibilidad hacia atrás. La incorporación de nuevas funcionalidades dentro de **Hartseer Analytics** se realizará mediante la extensión de los contratos existentes o la incorporación de nuevos endpoints, evitando modificaciones incompatibles que obliguen a actualizar los consumidores ya implementados. Esta estrategia permite que la API evolucione de forma progresiva, preservando la estabilidad de las integraciones existentes y garantizando una base sólida para futuras versiones del proyecto.

DISEÑO DE LOS ENDPOINTS

La API de **Hartseer Analytics** fue diseñada siguiendo una organización basada en dominios funcionales del negocio. En lugar de estructurar los recursos según tablas de la base de datos o componentes específicos del frontend, cada endpoint representa un área analítica independiente dentro de la aplicación, agrupando la información necesaria para responder a una necesidad concreta del proceso de toma de decisiones.

Esta organización permite mantener una clara separación entre los distintos ámbitos analíticos del negocio, facilitando tanto la evolución independiente de cada recurso como la incorporación de nuevas funcionalidades sin afectar la estructura general de la API. Al mismo tiempo, cada endpoint mantiene los principios arquitectónicos definidos durante el diseño, compartiendo contratos homogéneos, criterios de validación consistentes y una estructura común de respuestas.

El endpoint **Executive** fue concebido como el punto de acceso principal para el análisis global del negocio. Su propósito consiste en proporcionar una visión

consolidada del rendimiento general de la organización mediante indicadores estratégicos, tendencias temporales y métricas agregadas que permitan evaluar la evolución del negocio desde una perspectiva ejecutiva.

Por su parte, el endpoint **Products** fue diseñado para analizar el desempeño comercial del catálogo de productos desde distintas dimensiones de análisis. La incorporación de mecanismos de agrupamiento permite obtener información agregada por categorías, marcas o productos individuales, manteniendo un único contrato de respuesta independientemente de la dimensión seleccionada y favoreciendo la reutilización de la información por diferentes consumidores.

El endpoint **Customers** se orienta al análisis del comportamiento de la cartera de clientes durante el período seleccionado. Su diseño incorpora indicadores relacionados con adquisición, recurrencia, volumen de operaciones y rentabilidad, permitiendo evaluar la evolución de la base de clientes desde una perspectiva comercial sin depender de la representación visual utilizada posteriormente por el frontend.

Finalmente, el endpoint **Marketing** concentra toda la información vinculada al desempeño de los distintos canales comerciales. Su propósito consiste en facilitar la comparación del rendimiento entre canales mediante indicadores asociados a ingresos, rentabilidad, inversión publicitaria y retorno sobre la inversión, proporcionando una visión unificada del impacto generado por cada estrategia de comercialización.

La definición de estos cuatro dominios funcionales responde a la estructura analítica previamente implementada en el frontend y en los dashboards desarrollados con Power BI, permitiendo mantener coherencia entre las distintas etapas del proyecto. Sin embargo, la API fue diseñada de manera completamente independiente de dichas interfaces, representando entidades y conceptos propios del negocio en lugar de componentes visuales específicos.

Como consecuencia, cada endpoint constituye un servicio analítico reutilizable capaz de ser consumido por múltiples aplicaciones sin necesidad de modificaciones adicionales. Esta organización proporciona una base sólida para la incorporación de nuevos recursos en futuras versiones de **Hartseer Analytics**, preservando la consistencia arquitectónica y la compatibilidad con las integraciones existentes.